

Practical no:7

Aim: Association Rule Mining

Implement the Association Rule Mining algorithm (e.g., Apriori) to find frequent itemsets.

Generate association rules from the frequent itemsets and calculate their support and confidence.

Interpret and analyze the discovered association rules.

```
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules
from mlxtend.preprocessing import TransactionEncoder

df = pd.read_csv("iris.csv")
df = df.drop(columns=["Id"])

def bin_feature(series, name):
    labels = [f"{name}=Low", f"{name}=Medium", f"{name}=High"]
    return pd.qcut(series, q=3, labels=labels)

df["SL"] = bin_feature(df["SepalLengthCm"], "SepalLength")
df["SW"] = bin_feature(df["SepalWidthCm"], "SepalWidth")
df["PL"] = bin_feature(df["PetalLengthCm"], "PetalLength")
df["PW"] = bin_feature(df["PetalWidthCm"], "PetalWidth")
df["SP"] = "Species=" + df["Species"].str.replace("Iris-", "",
regex=False)

transactions = df[["SL", "SW", "PL", "PW",
"SP"]].astype(str).values.tolist()

te = TransactionEncoder()
te_ary = te.fit(transactions).transform(transactions)
df_enc = pd.DataFrame(te_ary, columns=te.columns_)

frequent_itemsets = apriori(df_enc, min_support=0.1, use_colnames=True)
frequent_itemsets["length"] = frequent_itemsets["itemsets"].apply(len)
frequent_itemsets = frequent_itemsets.sort_values("support",
ascending=False)

print(f"Total frequent itemsets found: {len(frequent_itemsets)}")
print()
```

```

print("Top 15 Frequent Itemsets:")
print(frequent_itemsets.head(15).to_string(index=False))
print()

rules = association_rules(frequent_itemsets, metric="confidence",
min_threshold=0.7)
rules = rules.sort_values(["confidence", "lift"], ascending=False)

print(f"Total rules generated: {len(rules)}")
print()
print("Top 15 Rules (by confidence):")
cols = ["antecedents", "consequents", "support", "confidence", "lift"]
print(rules[cols].head(15).to_string(index=False))
print()

species_rules = rules[rules["consequents"].apply(lambda x: any("Species"
in i for i in x))]
print(f"Rules predicting Species ({len(species_rules)}):")
print(species_rules[cols].head(10).to_string(index=False))

```

```

import pandas as pd

species_rules = rules[rules["consequents"].apply(lambda x: any("Species" in i for i in x))]
print(f"Rules predicting Species ({len(species_rules)}):")
print(species_rules[cols].head(10).to_string(index=False))

```

0.8s

Total frequent itemsets found: 117

support	itemsets	length
0.380000	(Sepalwidth=Low)	1
0.373333	(Sepallength=Medium)	1
0.360000	(Petalwidth=Medium)	1
0.346667	(Petalwidth=Low)	1
0.346667	(Sepallength=Low)	1
0.340000	(Sepallength=Medium)	1
0.333333	(Species=setosa)	1
0.333333	(Petalwidth=Low)	1
0.333333	(Petalwidth=Low)	1
0.333333	(Petalwidth=Low, Petallength=Low)	2
0.333333	(Species=versicolor)	1
0.333333	(Species=virginica)	1
0.333333	(Petalwidth=Low, Species=setosa)	2
0.333333	(Petalwidth=Low, Species=setosa)	2
0.333333	(Petalwidth=Low, Petallength=Low, Species=setosa)	3

Total rules generated: 328

Top 15 Rules (by confidence):

antecedents	consequents	support	confidence	lift
(Petalwidth=Low, Sepalwidth=Medium)	(Sepallength=Low, Species=setosa)	0.106667	1.0	3.333333
...	...	...	...	...
(Petalwidth=Low)	(Species=setosa)	0.333333	1.0	3.000000
(Petalwidth=Low, Petallength=Low)	(Species=setosa)	0.333333	1.0	3.000000
(Petalwidth=Low)	(Petalwidth=Low, Species=setosa)	0.333333	1.0	3.000000
(Petalwidth=Low)	(Petalwidth=Low, Species=setosa)	0.333333	1.0	3.000000

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

`!pip install mlxtend`